

PARSIM: Particles Simulation

José Brás
jose.bras@cern.ch
CERN
Geneva, Switzerland

Pedro Matias
pedro.matias@inesc-id.pt
INESC-ID
Lisbon, Portugal

João Moreira
joao.lobo.moreira@tecnico.ulisboa.pt
Instituto Superior Técnico
Lisbon, Portugal

Abstract

Simulating large-scale particle interactions under gravitational and collision dynamics is computationally intensive, requiring efficient parallelization to achieve high performance. In this work, we present an optimized OpenMP-based parallel implementation of the *n-body* problem in a particle simulation, focusing on efficient domain decomposition, reduced synchronization overhead, and collision detection methods. A key contribution is the integration of a Sweep-and-Prune algorithm for broad-phase collision detection, which significantly reduces the number of pairwise checks by sorting particles along one axis and pruning unnecessary comparisons. Additionally, we employ a Breadth-First Search (BFS)-based approach to efficiently resolve chained collisions, preventing deep recursive calls, and improving performance in densely packed regions. These optimizations, combined with a spatial flattened matrix decomposition for load balancing and fine-grained cell locking for safe particle migration, lead to substantial performance gains over serial implementation. Our benchmark results show near-linear speedup on multi-core processors and significant reductions in execution time.

José Brás, Pedro Matias, and João Moreira. 2025. **PARSIM**: Particles Simulation. 4 pages.

1 Introduction

Simulating the interactions of a large number of particles in the *n-body*[3] problem under gravity with collision dynamics is a computationally intensive task. Each simulation time-step involves calculating gravitational forces between many particle pairs, updating particle velocities and positions, and detecting collisions among particles in close proximity. The cost grows quickly with the number of particles, making a straightforward serial implementation very slow for large simulations. To address this, we employ parallelization on shared-memory multi-core processors using OpenMP [2]. The motivation is to exploit data parallelism inherent in the problem (many particles and spatial regions can be processed independently) to significantly speed up the simulation. Our goal is to optimize the simulation code by restructuring it into parallel phases, minimizing synchronization overhead, and ensuring good load balancing across threads. This report presents our parallelization strategy, the domain decomposition used, key optimizations for

performance, synchronization considerations, and an overview of results comparing the optimized parallel version to a serial baseline.

2 Algorithm Overview

We restructured the simulation into distinct phases that can be executed in parallel by multiple threads. Each phase corresponds to a computational step in the simulation time-step, and operates largely on independent portions of data. The main phases, as show in Figure 1 are: (1) Grid computation, (2) Gravitational force calculation, (3) Velocity update, (4) Grid recomputation, and (5) Collision detection/processing. A fixed team of threads is launched once at the start of the simulation and remains active throughout all time-steps, eliminating the overhead of repeated thread creation. The threads execute each phase in parallel, then synchronize using barriers before moving to the next phase to preserve data dependencies.

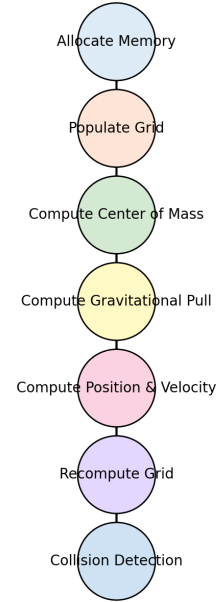


Figure 1: Dependency graph algorithm

3 Parallelization and Decomposition

We decided to parallelize the grid using a flattened grid decomposition. By representing the $N \times N$ grid as a one-dimensional array of $N \times N$ cells, we improve cache locality, which can even lead to superlinear speedups in some cases. In this approach, each thread is assigned a contiguous block of rows from the flattened grid, ensuring an even workload distribution. The decomposition

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

illustrated below shows how the grid is partitioned into contiguous segments, with each segment assigned to a separate thread.

Flattened 1×9 Grid Partitioning

Thread 0	Thread 1	Thread 2	Thread 3
0	1	2	3
4	5	6	7
8			

3.1 Partitioning

We partition the simulation domain by cells rather than by individual particles to improve cache locality and reduce synchronization overhead. By assigning whole cells to each thread, most interactions are handled entirely within one thread, minimizing cross-thread interference. Only when particles move between cells do we require fine-grained locks.

3.2 Load Balancing

Our flattened submatrix decomposition naturally balances the workload. For an $N \times N$ grid, dividing the domain evenly among threads ensures that each thread processes a similar number of cells (and thus particles), even in non-uniform distributions. Given n_{threads} , the simulation domain is represented as a flattened grid where the total number of cells is

$$\text{total_cells} = \text{ncside} \times \text{ncside}.$$

To evenly distribute these cells among threads, we calculate the number of cells each thread should handle using ceiling division:

$$\text{cells_per_thread} = \frac{\text{total_cells} + n_{\text{threads}} - 1}{n_{\text{threads}}}.$$

Within the parallel region, each thread computes its assigned range of cells as follows:

- $\text{idx_start} = \text{tid} \times \text{cells_per_thread}$ is the starting index for the thread.
- $\text{idx_end} = \min(\text{total_cells}, \text{idx_start} + \text{cells_per_thread})$ ensures that the ending index does not exceed the total number of cells.

This strategy assigns each thread a contiguous block of the flattened cell array, leading to balanced work distribution and improved cache locality.

The following code snippet illustrates this partitioning approach:

```
#pragma omp parallel
{
    // Compute this thread's linear cell range
    int32_t tid = omp_get_thread_num();
    int32_t total_cells = ncside * ncside;
    int32_t cells_per_thread =
        (total_cells + n_threads - 1) / n_threads;
    int32_t idx_start = tid * cells_per_thread;
    int32_t idx_end =
        std::min(total_cells,
            idx_start + cells_per_thread);
}
```

4 Synchronization

We enforce data dependencies between simulation phases using minimal synchronization. At the end of each major phase (e.g., center-of-mass computation, force calculation, and position/velocity updates), an OpenMP barrier ensures that all threads finish before proceeding. Fine-grained locks are used only when updating shared cells during particle migration, preventing race conditions with minimal overhead.

To reduce scheduling overhead, we statically decompose the work among threads. Each thread computes its own indices and processes its assigned cells without relying on dynamic scheduling. This avoids per-iteration assignment costs and minimizes synchronization, as barriers are only inserted at phase boundaries. As a result, threads execute largely independently within each phase, yielding improved scaling.

5 Collision Detection

Collision detection is computationally intensive due to the many pairwise comparisons needed to identify colliding particles. In our approach, we use a two-step method to mitigate this cost. First, the sweep-and-prune algorithm sorts particles by their x -coordinate and limits collision checks to those within a narrow x -window, reducing the number of comparisons from $O(n^2)$ to $O(n \log n)$ per subdomain (Algorithm 1). Second, to avoid the drawbacks of recursion (such as deep recursion or stack overflow), we implement an iterative breadth-first search (BFS) to group colliding particles. Since collision chains are typically short, the BFS overhead is minimal. Together, these techniques yield a robust and efficient collision detection process (Algorithm 2).

6 Performance Evaluation

This section presents the performance results of our optimized parallel implementation compared to the original serial version. We report execution times and speedups for various thread counts on a shared-memory multi-core system (the Lab2 machines), using 1, 2, 4, and 8 threads in OpenMP, with the single-thread (serial) case serving as the baseline. Identical simulation scenarios were used for all runs to ensure a fair comparison. This information can be found in the Tables 1 and 2 in the page below.

We observed that speedup varies with problem size, particle distribution, grid decomposition, and grid size. For a single OpenMP thread, the speedup is marginal (best case 1.03) because the serial version was highly optimized. With 2 threads, the best speedup reached 1.87 for simulations with 5000 or 10000 timesteps, while the worst case (4 timesteps) achieved only a 1.05 speedup.

For 4 threads, we achieved superlinear speedup in one instance (10 timesteps, grid size 20), attributed to enhanced cache utilization from our flat grid decomposition. More typical speedups were 3.49 and 3.45 for 500 and 10000 timesteps, respectively, on uniform distributions. The lowest speedup for 4 threads occurred with 1000 timesteps and a grid size of 30 in a normal distribution, likely because many cells in the flat array were inactive.

For 8 threads, note that the Lab2 machines have only 6 physical cores (12 with hyperthreading), so the observed speedups are closer to 6. Our best speedup with 8 threads was 6.33 (10 timesteps, 20×20 grid on a uniform distribution), while the worst occurred for a 3×3

Algorithm 1: Detect Collisions

Input: grid, row_start, row_end, col_start, col_end
for row $\leftarrow$ row_start **to** row_end - 1 **do**
 for col $\leftarrow$ col_start **to** col_end - 1 **do**
 cell $\leftarrow$ grid[row][col];
 numParticles $\leftarrow$ size(cell.particles);
 if numParticles < 2 **then**
 continue;
 // Define collision constants
 COLLISION_RADIUS $\leftarrow 5 \times 10^{-3}$;
 thresh2 $\leftarrow$ COLLISION_RADIUS $\times$
 COLLISION_RADIUS;
 // Sort particles by x-coordinate
 Sort cell.particles by x-coordinate;
 for i $\leftarrow$ 0 **to** numParticles - 1 **do**
 pA $\leftarrow$ cell.particles[i];
 ax $\leftarrow$ pA.x, ay $\leftarrow$ pA.y;
 for j $\leftarrow$ i + 1 **to** numParticles - 1 **do**
 pB $\leftarrow$ cell.particles[j];
 dx $\leftarrow$ pB.x - ax;
 if dx > COLLISION_RADIUS **then**
 break;
 dy $\leftarrow$ |ay - pB.y|;
 if dy > COLLISION_RADIUS **then**
 continue;
 dist2 $\leftarrow$ dx² + dy²;
 if dist2 $\leq$ thresh2 **then**
 pA.add_colliding(pB);
 pB.add_colliding(pA);

grid on a uniform distribution, where significant locking overhead was observed due to updates to the global flat matrix.

Overall, our simulation exhibits consistent speedups across varying problem sizes, particle distributions, timesteps, and grid sizes, demonstrating the robustness and scalability of our parallel implementation.

7 Conclusion

This section will summarize the key takeaways and suggest future improvements. In conclusion, we successfully optimized a particle simulation with gravitational interactions and collisions using OpenMP parallelization. By carefully structuring the computation into parallel phases and using a static flattened matrix decomposition of the domain, we achieved substantial speedups over the serial implementation. The final parallel program maintains correctness (producing the same physical results as the serial version) while running significantly faster by utilizing multiple cores. Key factors in our success were minimizing synchronization overhead (through merged computations and barriers only where needed) and employing efficient algorithms for collision detection and load balancing. There are several avenues for future work, we plan to

Algorithm 2: Compute collisions

Input: Grid, row_start, row_end, col_start, col_end
Output: collision_count, groups
for row $\leftarrow$ row_start **to** row_end - 1 **do**
 for col $\leftarrow$ col_start **to** col_end - 1 **do**
 cell $\leftarrow$ grid[row][col];
 if size(cell.particles) < 2 **then**
 continue;
 for p_ix $\leftarrow$ size(cell.particles)-1 **to** 0 **do**
 par $\leftarrow$ cell.particles[p_ix];
 if par.colliding_particles is empty **then**
 continue;
 cell.remove_particle(p_ix);
 par.reset_mass();
 if par $\in$ visited **then**
 continue;
 group $\leftarrow$ empty list;
 Enqueue(par) into queue q;
 Add par to visited;
 while q is not empty **do**
 current $\leftarrow$ Dequeue(q);
 Append current to group;
 foreach neighbor in
 current.colliding_particles **do**
 if neighbor $\notin$ visited **then**
 Add neighbor to visited;
 Enqueue(neighbor) into q;
 collision_count $\leftarrow$ collision_count + 1;
 groups.append(group);

transition from our shared memory model to a distributed memory paradigm using MPI [1]. By leveraging our flattened matrix decomposition—where each row can be independently assigned to a compute node—we expect to achieve higher scalability and performance, especially for large-scale problems. Future work will focus on optimizing data distribution, inter-node communication, and load balancing across heterogeneous resources.

References

- [1] Mpi: A message-passing interface standard, version 3.1. <https://www.mpi-forum.org/docs/>, 2015. Accessed: Mar. 2025.
- [2] Openmp application programming interface, version 5.0. <https://www.openmp.org/specifications/>, 2020. Accessed: Mar. 2025.
- [3] Wikipedia contributors. N-body problem. https://en.wikipedia.org/wiki/N-body_problem. Accessed: Mar. 2025.

Table 1: Execution Times for Serial and OMP Versions

Test	Command	Serial (s)	OMP 1 (s)	OMP 2 (s)	OMP 4 (s)	OMP 8 (s)
1	./parsim 1 5000 100 1000000 4	2.1	2.4	2.0	0.8	0.7
2	./parsim 1 5000 100 1000000 100	49.7	57.1	46.4	16.9	15.5
3	./parsim 1 5000 20 1000000 10	142.6	138.1	85.3	28.7	22.5
4	./parsim 1 1000 3 10000 10000	385.6	388.0	281.6	239.4	197.3
5	./parsim 3 5000 50 1000000 300	437.3	460.7	251.7	125.3	107.2
6	./parsim 3 5000 50 1000000 500	727.9	756.0	390.7	210.5	172.0
7	./parsim -1 1000 30 100000 1000	118.3	124.3	63.2	60.8	51.0
8	./parsim 12672 0.05 3 10 10	0.0	0.0	0.0	0.0	0.0
9	./parsim 5893 0.05 3 10 10	0.0	0.0	0.0	0.0	0.0
10	./parsim 8555 0.05 3 10 10	0.0	0.0	0.0	0.0	0.0
11	./parsim 12 100 5 10000 10000	70.0	81.0	44.1	26.9	23.3
12	./parsim -11 3500 20 500000 10	132.9	128.1	63.4	62.4	64.1

Table 2: Speedups from Serial to OMP Versions

Test	Command	OMP 1	OMP 2	OMP 4	OMP 8
1	./parsim 1 5000 100 1000000 4	0.87	1.05	2.625	3.00
2	./parsim 1 5000 100 1000000 100	0.87	1.07	2.94	3.20
3	./parsim 1 5000 20 1000000 10	1.03	1.67	4.96	6.33
4	./parsim 1 1000 3 10000 10000	0.99	1.67	1.61	1.95
5	./parsim 3 5000 50 1000000 300	0.95	1.73	3.49	4.07
6	./parsim 3 5000 50 1000000 500	0.96	1.86	3.45	4.23
7	./parsim -1 1000 30 100000 1000	0.95	1.87	1.94	2.31
8	./parsim 12672 0.05 3 10 10	0	0	0	0
9	./parsim 5893 0.05 3 10 10	0	0	0	0
10	./parsim 8555 0.05 3 10 10	0	0	0	0
11	./parsim 12 100 5 10000 10000	0.86	1.59	2.60	3.00
12	./parsim -11 3500 20 500000 10	1.03	2.09	2.12	2.07